



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC-2685 ROBÓTICA MÓVIL

LAB 1: OPERACIONES BÁSICAS DE MOVIMIENTO Y PERCEPCIÓN EN TURTLEBOT

Fecha de entrega: lunes 28 de Septiembre de 2026

Conociendo a TurtleBot

En este primer laboratorio aprenderemos y pondremos en práctica las destrezas básicas de movimiento y percepción en los TurtleBots. El objetivo de esta experiencia es programar herramientas de software que permitan al robot moverse en el espacio y percibir la configuración del entorno a través de la detección de obstáculos.

Antes de comenzar sus desarrollos, tengan presente que sus programas deben tener una estructura modular que haga posible identificar rápidamente posibles fuentes de errores. Por esta razón, antes de programar, es importante diseñar cada nodo definiendo las tareas que deberá realizar (ej: percepción, movimiento, etc.), así como también, los datos de entrada y salida (ej: image, distancia, comando de velocidad, etc.).

Para realizar sus desarrollos, usted deberá crear el paquete `lab1` dentro de un Workspace de ROS2, y almacenar dentro de él todos sus archivos (nodos, bibliotecas, lanzadores, etc.). Durante los laboratorios, usted podrá elegir el lenguaje de programación que más le acomode: Python (3.10+) o C++.

El día de la entrega, cada grupo tendrá un máximo de 40 minutos para presentar sus diseños, resultados y demostraciones prácticas. Para ello, se enviará con anterioridad un formulario con los bloques disponibles, los que serán asignados por orden de inscripción. Además, tanto las slides como los paquetes desarrollados para resolver este laboratorio, deberán ser enviados en un archivo comprimido a través de Canvas.

Dentro de sus slides deberán incluir como mínimo: un diagrama con el diseño del software a nivel de nodos, los resultados de las mediciones solicitadas en el presente enunciado y sus conclusiones. El día de la presentación, el profesor elegirá de forma aleatoria a un miembro del grupo para presentar las slides, y a otro para hacer la demostración práctica, por lo tanto, cada integrante deberá conocer en detalle ambas secciones.

1. Programando movimientos de TurtleBot

En esta actividad, usted deberá programar su robot para que sea capaz de alcanzar una **secuencia de poses** definidas dentro de un archivo de configuración. Para lograrlo, usted deberá construir un nodo que calcule y aplique los comandos de velocidad necesarios para llegar a cada objetivo. Dichos comandos de velocidad, deberán ser publicados en el siguiente tópico:

- `/cmd_vel`: tópico desde donde el robot recibe comandos de velocidad. La estructura de datos utilizada es de tipo *Twist*.

En particular, se solicita crear un nodo llamado `dead_reckoning_nav`, el cual implementará un algoritmo de navegación basado en una técnica conocida como *navegación por estima* (o *dead reckoning* en inglés) [1]. Esta antigua técnica, permite realizar una estimación de la posición actual del móvil en base a: i) el punto de partida, ii) la dirección de movimiento, y iii) las velocidades aplicadas. Según esta metodología, si deseamos que el robot se mueva una distancia d al aplicar una velocidad constante v , dicho comando de velocidad deberá ser ejecutado durante un tiempo igual a: $t = d/v$.

Para implementar un código ordenado y legible, se recomienda separar cada tarea de acuerdo a su **nivel de abstracción**, tal como se ejemplifica en las siguiente funciones:

1. Primer Nivel: Función `aplicar_velocidad( speed_command_list )`:

Función que recibe como argumento una lista de comandos de velocidad (`speed_command_list`), y las aplica de forma secuencial en el robot. Cada comando estará formado por un vector de 3 dimensiones que especificará: i) la velocidad lineal v_i , ii) la velocidad angular ω_i , y iii) el tiempo de aplicación de sus respectivas velocidades t_i .

Según lo anterior, la lista de comandos de velocidad tomará la siguiente forma: $[..., (v_i, \omega_i, t_i), ...]$.

En la figura 1, se muestra un ejemplo de como alcanzar la pose de destino (x_1, y_1, θ_1) desde la pose de origen (x_0, y_0, θ_0) , a través de una lista de velocidades y tiempos. En este caso, se aplicará la siguiente secuencia de velocidades:

- 1) $(v_0, 0, t_0)$: velocidad lineal v_0 durante t_0 [s],
- 2) $(0, \omega_1, t_1)$: velocidad angular ω_1 durante t_1 [s],
- 3) $(v_2, 0, t_2)$: velocidad lineal v_2 durante t_2 [s].

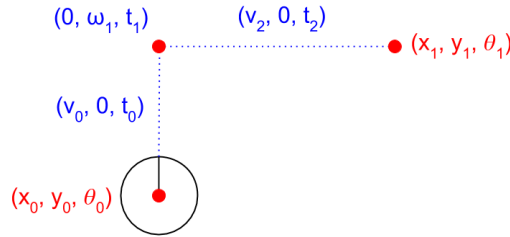


Figura 1: Lista de velocidades (lineal y angular) y tiempos de aplicación para ir de la pose (x_0, y_0, θ_0) a la pose (x_1, y_1, θ_1) .

2. Segundo Nivel: Función `mover_robot_a_destino( goal_pose )`:

Función que mueve el robot a una pose de destino, relativa a la pose actual. El argumento `goal_pose` corresponde a una pose en formato (x, y, θ) . A partir de este argumento, se deberá calcular una lista de comandos de velocidad en formato $[..., (v_i, \omega_i, t_i), ...]$, que permitan al robot alcanzar la pose objetivo (se recomienda fuertemente diseñar trayectorias en líneas rectas, tal como se ejemplifica en la figura 1). Por simplicidad, en esta experiencia utilizaremos una velocidad lineal constante de 0.2 [m/s], y una velocidad rotacional constante de 1.0 [rad/s]. Una vez calculada la lista de comandos de velocidad, use la función `aplicar_velocidad` para ejecutarlas.

3. Tercer Nivel: Función `accion_mover_cb`:

Esta función debe actuar como *callback* del subscritor al tópico `goal_list`. El mensaje recibido en dicho tópico corresponderá a una lista de poses que el robot deberá alcanzar de forma secuencial. Para ello, la función `accion_mover_cb` deberá iterar sobre la lista recibida, y pasar cada uno de sus ítems a la función `mover_robot_a_destino`. Se recomienda que la lista de poses recibida a través del tópico `goal_list` sea transportada a través de un mensaje de tipo `geometry_msgs/PoseArray`.

Adicionalmente, usted deberá implementar un nodo cliente llamado `pose_loader`, que deberá leer la lista de poses desde un archivo de texto, para luego enviarla al nodo `dead_reckoning_nav` a través del tópico `goal_list`. La figura 2 ejemplifica la interacción entre el nodo `dead_reckoning_nav` y el cliente `pose_loader`.

Una vez programado el nodo `dead_reckoning_nav`, deberá realizar las siguientes actividades y tomar las mediciones que se solicitan.

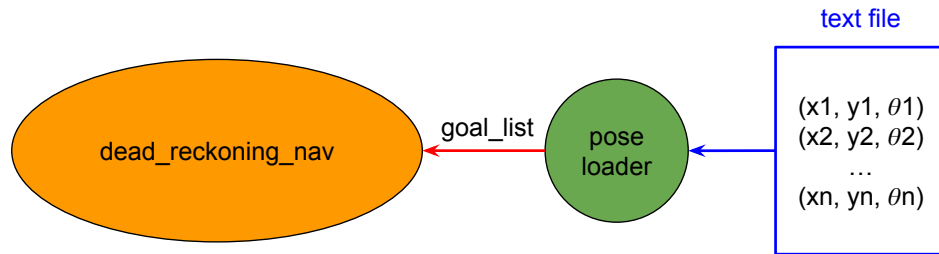


Figura 2: Interacción entre nodos `deadreckoning` y cliente `pose_loader` para el envío de lista de poses.

Actividad 1.1: Prueba Funcional

Demostración: El profesor le entregará una lista de poses que usted deberá ingresar en su archivo de configuración. Posteriormente, usted deberá ejecutar su código para comprobar que el robot logra alcanzar cada pose de forma adecuada.

Actividad 1.2: Avanzar y Rotar

Usando los nodos `deadreckoning` y `pose_loader`, configure una lista de poses que haga a su robot describir un cuadrado de lados 1 [m], completando esta figura tres veces seguidas (3 vueltas completas). Para ello, deberá editar el archivo de configuración y agregar las poses que definan cada uno de los vértices del cuadrado (replicando la secuencia 3 veces, una para cada cuadrado).

Posteriormente, levante sus nodos para que el robot empiece a seguir la trayectoria cuadrada. Mientras el robot realiza el movimiento, registre las coordenadas (x, y) reales y de odometría, que va siguiendo el robot en su trayectoria. Para obtener dichas coordenadas, usted deberá leer los tópicos `/real_pose` y `/odom` respectivamente.

A partir de los datos registrados en el experimento anterior:

- Construya un gráfico bidimensional que muestre la trayectoria seguida por el robot (visto desde arriba).
- Calcule el error promedio entre el punto de partida y llegada, considerando 5 experimentos de tres vueltas cada uno.
- Intente encontrar un factor de corrección para el tiempo calculado en la función *mover_robot_a_destino* ($t_{corrected} = t * factor$), de manera tal que el robot logre terminar la rutina lo más cerca posible del punto de partida (con un error de menos de 10 [cm]). Este factor deberá ser aplicado solo sobre los movimientos rotacionales.
- Aplique el factor de corrección encontrado y construya un gráfico bidimensional que muestre la trayectoria del robot.
- Aplicando el factor de corrección, calcule error promedio entre el punto de partida y llegada, considerando 5 experimentos de tres vueltas cada uno.

Incluir en presentación: i) gráfico de trayectoria de robot para movimientos sin factor de corrección, ii) Error promedio entre punto de partida y llegada sin factor de corrección, iii) gráfico de trayectoria de robot para movimientos con factor de corrección, iv) error promedio entre punto de partida y llegada con factor de corrección, v) explicar fenómeno observado y reflexionar acerca de las causas que pueden producirlo (hay razones de software y de hardware).

Demostración: Movimiento en cuadrado con factor de corrección.

Incluir en paquete: archivo launch de nombre *avanzar_y_rotar.xml* que permita iniciar los nodos que implementan su rutina y los nodos que implementan el simulador. Deben garantizar que sea posible correr el experimento completo simplemente ejecutando:

```
ros2 launch <nombre_paquete> avanzar_y_rotar.xml
```

2. Teleoperación de TurtleBot

En esta actividad, usted deberá desarrollar un nodo que permita mover el TurtleBot a través de las teclas del computador. Para ello, su nodo deberá detectar las teclas que están siendo presionadas, y según el carácter recibido, enviar comandos de velocidad al tópico `/cmd_vel` por medio de un mensaje de tipo `Twist`.

Para estandarizar el control de sus robots, usted deberá realizar la siguiente asignación de teclas a cada movimiento:

- tecla “i”: robot se mueve en línea recta a 0.2 [m/s]
- tecla “j”: robot se mueve en línea recta a -0.2 [m/s]
- tecla “a”: robot rota a 1.0 [rad/s]
- tecla “s”: robot rota a -1.0 [rad/s]
- tecla “q”: robot se mueve a una velocidad lineal de 0.2 [m/s] y rotacional de 1.0 [rad/s]
- tecla “w”: robot se mueve a una velocidad lineal de 0.2 [m/s] y rotacional de -1.0 [rad/s]

Demostración: Debe levantar su nodo de control y mostrar al profesor el correcto funcionamiento del programa.

Incluir en paquete: archivo launch de nombre *teleop.xml* que permita iniciar el nodo de control por teclado. Debe garantizar que sea posible correr el experimento completo simplemente ejecutando:

```
ros2 launch <nombre_paquete> teleop.xml
```

3. Dotando al TurtleBot de percepción básica

Hasta ahora, el nodo `dead_reckoning_nav` no tiene la capacidad de reaccionar al mundo que lo rodea. En esta parte dotaremos a los TurtleBots de un módulo de percepción básico usando el sensor *Kinect*, el cual se basará en la detección de obstáculos usando la imagen de profundidad.

El tópico a utilizar en esta sección es:

- `/camera/depth/image_raw`: tópico en el cual se publica la imagen de profundidad del sensor *Kinect*.

Para esta parte deberá programar el siguiente nodo:

obstacle_detector Usando la imagen de profundidad, el nodo debe publicar en forma continua la ausencia o presencia de algún obstáculo que pueda interferir en el avance del robot, junto con su ubicación considerando 3 regiones: izquierda, centro, derecha. El nodo `obstacle_detector` deberá suscribirse al tópico de la imagen de profundidad `/camera/depth/image_raw`, y publicar la detección de cada obstáculo detectado en el tópico de nombre `/occupancy_state`. Para codificar la detección por región, utilizaremos un vector de 3 dimensiones que representarán el estado de ocupación izquierdo, central o derecho respectivamente. Dentro de este vector, el estado *ocupado* deberá representarse por un 1, y el estado *desocupado*, con un 0. A modo de ejemplo, si el robot detecta un obstáculo a su derecha, el vector deberá tomar el valor: (0, 0, 1).

Para enviar el estado de ocupación al tópico `/occupancy_state`, se recomienda utilizar el mensaje `geometry_msgs/Vector3`. Para considerar un objeto como un obstáculo, éste se deberá encontrar a una distancia menor o igual a 50 [cm] de la punta del robot. Además, se recomienda encarecidamente usar funciones de la librería *NumPy* para el procesamiento eficiente de las imágenes.

3.1. Uniendo acción y percepción

Para lograr el objetivo final de este laboratorio, uniremos acción y percepción para lograr evitar colisiones. Para ello deberá modificar el nodo `dead_reckoning_nav` y agregar la capacidad de recibir mensajes de detección de obstáculos desde el tópico `/occupancy_state`. Al recibir un estado del tipo $(0, 0, 0)$, el robot deberá seguir su trayectoria de forma normal. Por otra parte, si el nodo `dead_reckoning_nav` recibe un mensaje desde el nodo `obstacle_detector` indicando la presencia de un obstáculo (al menos una dimensión del vector igual a 1), deberá detener el movimiento del robot e indicar con un log (mensaje en terminal) en que posición se encuentra el obstáculo (*obstacle left*, *obstacle right* u *obstacle center*). Una vez que el obstáculo sea retirado, el robot deberá continuar con su movimiento. Se recomienda modificar la función `aplicar_velocidad` para *congelar* su operación al detectar algún obstáculo, y luego reanudarla al detectar el camino libre.

Incluir en presentación: breve explicación del criterio utilizado para la detección.

Demostración: debe agregar y eliminar paredes mostrando que el robot se detiene y avanza respectivamente. Cada vez que el robot detecte un obstáculo, se deberá escribir en la terminal su posición (izquierda, central o derecha). Para el movimiento del robot, puede utilizar la trayectoria cuadrada de la actividad 1.

Incluir en paquete: archivo launch de nombre *accion_y_percepcion.xml* que permita iniciar tanto los nodos que implementan el comportamiento para evitar colisiones como los nodos que implementan el simulador. Deben garantizar que sea posible correr el experimento completo simplemente ejecutando:

```
ros2 launch <nombre_paquete> accion_y_percepcion.xml
```

Referencias

[1] https://en.wikipedia.org/wiki/Dead_reckoning

[2] https://github.com/gasevi/very_simple_robot_simulator